

Integración con FastAPI

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

-  Integrar SQLAlchemy con FastAPI usando dependencies
-  Crear endpoints CRUD conectados a base de datos
-  Usar Pydantic para validación y SQLAlchemy para persistencia
-  Manejar errores de base de datos en endpoints

Contenido

1. Dependency Injection para Database

FastAPI usa **Dependency Injection** para proveer la sesión de base de datos:

```
# database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker, Session
from typing import Generator

SQLALCHEMY_DATABASE_URL = "sqlite:///./app.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False} # Solo para SQLite
)

SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

class Base(DeclarativeBase):
    pass

def get_db() -> Generator[Session, None, None]:
    """
    Dependency que provee una sesión de base de datos.

    - Crea una nueva sesión para cada request
    - Cierra la sesión al finalizar (incluso si hay error)
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

2. Estructura de Archivos

```
src/
├─ main.py          # FastAPI app y endpoints
├─ database.py      # Configuración SQLAlchemy
```

```
└─ models.py      # Modelos SQLAlchemy (tablas)
└─ schemas.py     # Schemas Pydantic (validación)
```

3. Modelos SQLAlchemy vs Schemas Pydantic

Importante: Usamos modelos diferentes para cada propósito:

Archivo	Propósito	Librería
models.py	Definir tablas en DB	SQLAlchemy
schemas.py	Validar request/response	Pydantic

```
# models.py - Tabla en base de datos
from datetime import datetime
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column
from database import Base

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    hashed_password: Mapped[str] = mapped_column(String(255)) # ¡No exponer!
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

```
# schemas.py - Validación de datos
from datetime import datetime
from pydantic import BaseModel, EmailStr, Field, ConfigDict

class UserBase(BaseModel):
    """Campos comunes"""
    name: str = Field(..., min_length=2, max_length=100)
    email: EmailStr

class UserCreate(UserBase):
    """Schema para crear usuario"""
    password: str = Field(..., min_length=8)

class UserUpdate(BaseModel):
    """Schema para actualizar usuario (todos opcionales)"""
    name: str | None = Field(None, min_length=2, max_length=100)
    email: EmailStr | None = None
    is_active: bool | None = None

class UserResponse(UserBase):
    """Schema de respuesta (sin password)"""
    id: int
```

```

is_active: bool
created_at: datetime

# Permite crear desde objeto SQLAlchemy
model_config = ConfigDict(from_attributes=True)

class UserListResponse(BaseModel):
    """Schema para lista paginada"""
    users: list[UserResponse]
    total: int
    skip: int
    limit: int

```

4. CRUD Completo con FastAPI

```

# main.py
from fastapi import FastAPI, Depends, HTTPException, status, Query
from sqlalchemy import select, func
from sqlalchemy.orm import Session
from sqlalchemy.exc import IntegrityError

from database import engine, Base, get_db
from models import User
from schemas import UserCreate, UserUpdate, UserResponse, UserListResponse

# Crear tablas al iniciar
Base.metadata.create_all(bind=engine)

app = FastAPI(title="Users API", version="1.0.0")

# =====
# CREATE
# =====

@app.post(
    "/users",
    response_model=UserResponse,
    status_code=status.HTTP_201_CREATED,
    tags=["users"]
)
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    """Crea un nuevo usuario"""
    # Verificar si email ya existe
    stmt = select(User).where(User.email == user.email)
    existing = db.execute(stmt).scalar_one_or_none()

    if existing:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=f"Email {user.email} already registered"
        )

    # Crear usuario (en producción: hashear password!)

```

```

db_user = User(
    name=user.name,
    email=user.email,
    hashed_password=f"hashed_{user.password}" # ⚠ Solo ejemplo
)

db.add(db_user)
db.commit()
db.refresh(db_user)

return db_user

# =====
# READ
# =====

@app.get(
    "/users",
    response_model=UserListResponse,
    tags=["users"]
)
def list_users(
    skip: int = Query(0, ge=0, description="Offset for pagination"),
    limit: int = Query(10, ge=1, le=100, description="Limit results"),
    is_active: bool | None = Query(None, description="Filter by active status"),
    db: Session = Depends(get_db)
):
    """Lista usuarios con paginación y filtros"""
    # Query base
    base_stmt = select(User)

    # Aplicar filtros
    if is_active is not None:
        base_stmt = base_stmt.where(User.is_active == is_active)

    # Contar total
    count_stmt = select(func.count()).select_from(base_stmt.subquery())
    total = db.execute(count_stmt).scalar()

    # Aplicar paginación
    stmt = base_stmt.offset(skip).limit(limit).order_by(User.id)
    users = db.execute(stmt).scalars().all()

    return UserListResponse(
        users=users,
        total=total,
        skip=skip,
        limit=limit
    )

@app.get(
    "/users/{user_id}",
    response_model=UserResponse,
    tags=["users"]
)

```

```

)
def get_user(user_id: int, db: Session = Depends(get_db)):
    """Obtiene un usuario por ID"""
    user = db.get(User, user_id)

    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"User with id {user_id} not found"
        )

    return user

# =====
# UPDATE
# =====

@app.put(
    "/users/{user_id}",
    response_model=UserResponse,
    tags=["users"]
)
def update_user(
    user_id: int,
    user_update: UserUpdate,
    db: Session = Depends(get_db)
):
    """Actualiza un usuario existente"""
    user = db.get(User, user_id)

    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"User with id {user_id} not found"
        )

    # Actualizar solo campos proporcionados
    update_data = user_update.model_dump(exclude_unset=True)

    # Verificar email único si se está actualizando
    if "email" in update_data:
        stmt = select(User).where(
            User.email == update_data["email"],
            User.id != user_id
        )
        existing = db.execute(stmt).scalar_one_or_none()
        if existing:
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail=f"Email {update_data['email']} already in use"
            )

    for key, value in update_data.items():
        setattr(user, key, value)

```

```

        db.commit()
        db.refresh(user)

    return user

# =====
# DELETE
# =====

@app.delete(
    "/users/{user_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    tags=["users"]
)
def delete_user(user_id: int, db: Session = Depends(get_db)):
    """Elimina un usuario"""
    user = db.get(User, user_id)

    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"User with id {user_id} not found"
        )

    db.delete(user)
    db.commit()

    return None

# =====
# HEALTH CHECK
# =====

@app.get("/health", tags=["health"])
def health_check(db: Session = Depends(get_db)):
    """Verifica conexión a base de datos"""
    from sqlalchemy import text

    try:
        db.execute(text("SELECT 1"))
        return {"status": "healthy", "database": "connected"}
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail=f"Database unavailable: {str(e)}"
        )

```

5. El Flujo Completo

Request HTTP





6. Patrón de Separación (Servicios)

Para proyectos más grandes, separa la lógica en servicios:

```
# services/user_service.py
from sqlalchemy import select
from sqlalchemy.orm import Session
from models import User
from schemas import UserCreate, UserUpdate

class UserService:
    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, user_id: int) -> User | None:
        return self.db.get(User, user_id)

    def get_by_email(self, email: str) -> User | None:
        stmt = select(User).where(User.email == email)
        return self.db.execute(stmt).scalar_one_or_none()

    def create(self, user_data: UserCreate) -> User:
        user = User(
            name=user_data.name,
            email=user_data.email,
            hashed_password=f"hashed_{user_data.password}"
        )
```

```

        self.db.add(user)
        self.db.commit()
        self.db.refresh(user)
        return user

    def update(self, user: User, user_data: UserUpdate) -> User:
        update_dict = user_data.model_dump(exclude_unset=True)
        for key, value in update_dict.items():
            setattr(user, key, value)
        self.db.commit()
        self.db.refresh(user)
        return user

    def delete(self, user: User) -> None:
        self.db.delete(user)
        self.db.commit()

# En el endpoint
@app.post("/users", response_model=UserResponse)
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    service = UserService(db)

    if service.get_by_email(user.email):
        raise HTTPException(status_code=409, detail="Email already exists")

    return service.create(user)

```

7. Tips de Producción

Usar variables de entorno

```

# database.py
import os

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./app.db")

```

Logging de SQL

```

# Para debug
engine = create_engine(DATABASE_URL, echo=True)

# Para producción
engine = create_engine(DATABASE_URL, echo=False)

```

Manejo de errores global

```

from fastapi import Request
from fastapi.responses import JSONResponse
from sqlalchemy.exc import SQLAlchemyError

@app.exception_handler(SQLAlchemyError)
async def sqlalchemy_exception_handler(request: Request, exc: SQLAlchemyError):

```



```
return JsonResponse(  
    status_code=500,  
    content={"detail": "Database error occurred"}  
)
```

✓ Checklist

- ☐ Entiendo cómo funciona `Depends(get_db)`
- ☐ Sé la diferencia entre modelos SQLAlchemy y schemas Pydantic
- ☐ Puedo crear endpoints CRUD completos
- ☐ Uso `model_config = ConfigDict(from_attributes=True)` en schemas de respuesta
- ☐ Manejo errores con `HTTPException`

[← Anterior: Operaciones CRUD](#) | [Volver a README →](#)